

CURSOR STEPS PLAYBOOK

Organization Job Handler Application

Senior Prompt Engineer Guide for Cursor

Stack: React + Python (FastAPI) + PostgreSQL

Domain focus: Job CRUD, Job Problems, and data pipeline jobs
(Epicor → GD Warehouse Sync · TXT → RPT · RPT → Fabric · Data Flow → Lake House)

Method: Context-first · Plan before Agent · Incremental approvals · Domain-accurate prompts
Adapted from Ticket-Management curriculum — redesigned for Job Handler (not a blind copy)

Document type: Executable Cursor step roadmap with optimized prompts

Usage: Execute Phase → Review → Approve → Next Phase

How a Senior Prompt Engineer Uses This Playbook

Do not paste the entire playbook into one chat. Run one step (or one thin slice) per Agent session. After each step, review diffs, ask Cursor to explain non-obvious choices, then approve before continuing.

Operating rules

- Start every phase in Plan mode for architecture/requirements; switch to Agent only after the plan is accepted.
- Keep project rules (.cursor/rules), AGENTS.md, and a short domain glossary updated — that is your durable context.
- Prefer small, verifiable slices: schema → one API resource → one UI page → wire-up → tests.
- Name domain objects exactly: Job, JobProblem, JobRun, Pipeline (Epicor/TXT/RPT/Fabric/Lake House).
- After code generation, demand a walkthrough of files changed, risks, and how to run locally.
- Never ask for "build the whole app"; always constrain scope, acceptance criteria, and non-goals.

What changed vs the Excel schedule

- Express → Python FastAPI + SQLAlchemy/Alembic (your stack).
- Tickets → Jobs + Job Problems + pipeline JobTypes as first-class domain.
- Added Phase 0 context engineering (rules/AGENTS) — highest leverage PE work.
- Added explicit Plan-mode gates and approval pauses between slices.
- Collapsed redundant UI steps; added pipeline templates + run-status MVP instead of ticket-only screens.

Phases at a glance

- Phase 0 — Cursor Environment & Project Context Engineering
- Phase 1 — Domain Requirements, SRS, User Stories
- Phase 2 — Architecture & PostgreSQL Data Model
- Phase 3 — Python FastAPI Backend (Jobs & Job Problems CRUD)
- Phase 4 — React Frontend (Jobs Console UX)
- Phase 5 — Pipeline Job Types & Status / Orchestration UX
- Phase 6 — Quality: Review, Refactor, Tests, Debugging
- Phase 7 — Docs, Git Workflow, Docker, Deploy, CI/CD, Roadmap

PHASE 0 — Cursor Environment & Project Context Engineering

Step 1: Install Cursor and create the Job Handler workspace

Cursor topic: Installation & Setup

Why this step: Establish an isolated workspace so chat, rules, and indexing stay project-scoped.

Workspace setup

Optimized prompt (copy into Cursor)

- Create/open workspace at this Job Handler repo path only — do not mix unrelated projects.
- Confirm Node.js LTS, Python 3.11+, PostgreSQL client, and Git are available on PATH.
- List recommended Cursor settings for a monorepo with /frontend and /backend folders.
- Output a short checklist I can verify before Step 2.

→ Pause: review output, approve, then continue to the next step.

Step 2: Map the Cursor interface to this project workflow

Cursor topic: Cursor UI Overview (Chat, Composer, Agent, Plan, Inline Edit)

Why this step: Senior PEs assign tools deliberately — Plan for design, Agent for multi-file work, Inline for surgical edits.

Tooling map for Job Handler

Optimized prompt (copy into Cursor)

- Explain when to use Plan vs Agent vs Inline Edit for this stack (React + FastAPI + PostgreSQL).
- Recommend a default workflow: Plan → approved plan → Agent thin slice → Inline polish.
- Call out risks of long Agent runs on multi-file backends/frontends without constraints.
- Produce a one-page cheat sheet tailored to Job Handler (no generic fluff).

→ Pause: review output, approve, then continue to the next step.

Step 3: Configure models and Agent modes for coding vs planning

Cursor topic: Model Selection & Configuration

Why this step: Use stronger models for architecture/reviews; faster models for repetitive scaffolding once patterns exist.

Model policy

Optimized prompt (copy into Cursor)

- Propose a model policy: planning/architecture, implementation, refactoring, and quick Q&A.;
- Enable useful features only (rules, indexing, terminal) without over-automation.
- Document when NOT to let Agent auto-run terminal commands (migrations, destructive SQL).
- Write this as a short "AI operating manual" section for AGENTS.md.

→ Pause: review output, approve, then continue to the next step.

Step 4: Create durable project rules and coding standards

Cursor topic: Project Rules (.cursor/rules) & AGENTS.md

Why this step: Rules are the highest-leverage PE lever — they prevent reminding architecture every chat.

Project rules content

Optimized prompt (copy into Cursor)

- Create .cursor/rules (and AGENTS.md) for Job Handler with: stack constraints, folder layout, naming, API error shape, Do/Do-not list.
- Mandate FastAPI + SQLAlchemy/Alembic + Pydantic + React (Vite) + TypeScript + PostgreSQL.
- Encode domain terms: Job, JobType (EPICOR_GD_WH_SYNC, TXT_TO_RPT, RPT_TO_FABRIC, DATAFLOW_TO_LAKEHOUSE), JobProblem, JobRun, status enums.
- Forbid inventing ticket-system terminology; this is a Job Handler, not a helpdesk.
- Require asking approval before large new folders, schema changes, or dependency additions.
- Keep rules concise and enforceable (prefer always/never over essays).

→ Pause: review output, approve, then continue to the next step.

Step 5: Initialize Git with a safe baseline

Cursor topic: Git Integration

Why this step: Version control early so Agent diffs stay reviewable and reversible.

Git baseline

Optimized prompt (copy into Cursor)

- Initialize git if needed; add .gitignore for Python, Node, .env, venv, dist, __pycache__.
- Do NOT commit secrets. Show what belongs in .env.example instead.
- Suggest trunk-based branch naming for feature slices (jobs-crud, schema, ui-dashboard).
- Stop after proposing the first commit message — wait for my approval before committing.

→ Pause: review output, approve, then continue to the next step.

PHASE 1 — Domain Requirements, SRS, User Stories

Step 6: Ingest product intent and ask clarifying questions

Cursor topic: AI Chat / Requirements Discovery

Why this step: Force precision before code — ambiguity is the #1 cause of rework in Agent sessions.

Discovery interview

Optimized prompt (copy into Cursor)

- Act as a senior BA + prompt engineer. Restate the product in your own words for Organization Job Handler.
- Cover actors (Admin, Operator, Viewer), CRUD on Jobs, Job Problems, run history, and the four pipeline examples.
- Ask sharp clarifying questions only (auth?, multi-tenant?, scheduling?, real Epicor connectivity vs simulated statuses?).
- Produce Assumptions vs Open Questions. Do not write code yet.

→ Pause: review output, approve, then continue to the next step.

Step 7: Produce a lean SRS for Job Handler

Cursor topic: Requirements Gathering (SRS)

Why this step: A typed SRS becomes the shared source of truth for later Agent prompts.

SRS deliverable

Optimized prompt (copy into Cursor)

- Generate a lean SRS (not a novel): purpose, scope/non-goals, functional requirements, non-functional, data entities, integrations.
- Include FR for: Job CRUD, JobProblem CRUD, list/filter by JobType and status, soft delete or archive policy.
- Describe pipeline jobs as first-class JobTypes with source→target semantics (Epicor→GD WH, TXT→RPT, RPT→Fabric, Data Flow→Lake House).
- Add acceptance-level test ideas per major FR. Save as docs/SRS.md. Wait for approval before architecture.

→ Pause: review output, approve, then continue to the next step.

Step 8: Derive user stories and acceptance criteria

Cursor topic: Business Analysis (User Stories)

Why this step: Stories are the right unit of Agent work; each later coding step should map to one story.

Stories & AC

Optimized prompt (copy into Cursor)

- Write user stories in As a... I want... So that... format for Admin/Operator/Viewer.
- Include AC with Given/When/Then for: create job, edit, delete/archive, attach problems, filter by type, view pipeline lineage.
- Prioritize MVP (MoSCoW). Mark MUST for CRUD + Job Problems + four JobTypes as metadata; SHOULD for live orchestration.
- Save as docs/user-stories.md. Do not scaffold code yet.

→ Pause: review output, approve, then continue to the next step.

PHASE 2 — Architecture & PostgreSQL Data Model

Step 9: Design solution architecture (Plan mode)

Cursor topic: Solution Architecture (Plan Mode)

Why this step: Architecture before folders prevents FastAPI/React coupling mistakes and chaotic monorepos.

Architecture plan

Optimized prompt (copy into Cursor)

- In Plan mode, design architecture for React + FastAPI + PostgreSQL Job Handler.
- Propose: monorepo layout, API versioning (/api/v1), auth approach for MVP, migration strategy (Alembic), env config.
- Draw ASCII or Mermaid: Client → API → Services → Repositories → PostgreSQL; plus JobType pipeline diagram.
- List risks (N+1 queries, unbounded job logs, secret handling). Get approval before generating folders.

→ Pause: review output, approve, then continue to the next step.

Step 10: Generate the approved folder structure only

Cursor topic: Agent Mode (scaffolding with constraints)

Why this step: Scaffold structure without business logic so boundaries stay clean.

Scaffold

Optimized prompt (copy into Cursor)

- Create ONLY the agreed folder structure and placeholder README stubs for /backend and /frontend.
- Backend outline: app/main.py, api/, models/, schemas/, services/, repositories/, core/config.py, alembic/.
- Frontend outline: src/pages, components, features/jobs, features/problems, api client, types.
- Do not invent unused microservices. No business logic yet. Show tree and stop for approval.

→ Pause: review output, approve, then continue to the next step.

Step 11: Design the PostgreSQL logical data model

Cursor topic: Database Design

Why this step: Correct entities (Job vs JobRun vs JobProblem) avoid rewriting APIs later.

Logical model

Optimized prompt (copy into Cursor)

- Design ER model: organizations (optional MVP), jobs, job_problems, job_runs (optional MVP), enums for job_type and status.
- Job fields: name, job_type, source_system, target_system, schedule_cron (nullable), status, config JSONB, created_at/updated_at.
- JobProblem fields: job_id FK, severity, code, message, occurred_at, resolved_at, metadata JSONB.
- Encode pipeline pairs for the four examples as constrained job_type + source/target. Output docs/data-model.md + Mermaid ER. No SQL yet.

→ Pause: review output, approve, then continue to the next step.

Step 12: Generate SQL / Alembic migrations

Cursor topic: SQL Generation & Migrations

Why this step: Migrations are the durable truth for PostgreSQL — keep them reviewable and reversible.

Migration slice

Optimized prompt (copy into Cursor)

- From the approved data model, generate Alembic initial migration + raw SQL reference in docs/schema.sql.
- Include indexes for (job_type), (status), (job_id on problems), and updated_at sorting.
- Add CHECK or enum types for job_type and status. Provide up/down notes.
- Do not run migrate against production; give local docker-compose postgres commands. Wait for approval.

→ Pause: review output, approve, then continue to the next step.

PHASE 3 — Python FastAPI Backend (Jobs & Job Problems CRUD)

Step 13: Bootstrap FastAPI application core

Cursor topic: Backend Bootstrap / Multi-file Code Generation

Why this step: Core config, DB session, health check, and error handlers must exist before CRUD routers.

FastAPI core

Optimized prompt (copy into Cursor)

- Implement FastAPI app endpoint, settings via pydantic-settings, SQLAlchemy engine/session, CORS for local React, /health.
- Standardize error response JSON: {detail, code, fields?} and request logging middleware (no secrets).
- Add requirements.txt / pyproject.toml and a .env.example. Keep dependencies minimal.
- Explain how to run uvicorn locally. Do not implement full CRUD yet.

→ Pause: review output, approve, then continue to the next step.

Step 14: Implement Jobs REST API (CRUD)

Cursor topic: Backend Development — Jobs resource

Why this step: Jobs are the aggregate root; stabilize this API before problems and UI.

Jobs API

Optimized prompt (copy into Cursor)

- Implement /api/v1/jobs CRUD with Pydantic schemas, service layer, and repository — no logic in routers.
- Support pagination, filter by job_type/status, search by name. Validate job_type enum for the four pipeline types (+ extensible OTHER if needed).
- Include OpenAPI-friendly models and clear 404/422 behavior. Add seed data for the four example jobs.
- Show curl examples for each endpoint. Pause for approval before Job Problems API.

→ Pause: review output, approve, then continue to the next step.

Step 15: Implement Job Problems REST API (CRUD)

Cursor topic: Backend Development — JobProblems nested resource

Why this step: Problems are first-class operational records linked to jobs; treat them as a proper resource.

Job Problems API

Optimized prompt (copy into Cursor)

- Implement /api/v1/jobs/{job_id}/problems and /api/v1/problems CRUD/list patterns as designed.
- Support severity filter, unresolved-only filter, resolve endpoint (PATCH) setting resolved_at.
- Enforce FK integrity and cascading rules per approved schema. Keep response shapes consistent with Jobs.
- Provide curl examples and a short sequence: create job → create problem → list → resolve.

→ Pause: review output, approve, then continue to the next step.

Step 16: Explain and stress-test generated backend

Cursor topic: Code Explanation & Backend Walkthrough

Why this step: Comprehension gates prevent shipping patterns you cannot maintain.

Walkthrough

Optimized prompt (copy into Cursor)

- Explain request flow for POST /jobs and POST /problems with file references.
- Call out where validation, transactions, and commit/rollback happen.
- List 3 likely failure modes (bad enum, missing FK, DB down) and how they surface.
- Suggest 5 manual test cases before frontend work. No new features.

→ Pause: review output, approve, then continue to the next step.

PHASE 4 — React Frontend (Jobs Console UX)

Step 17: Scaffold React + TypeScript (Vite) app

Cursor topic: Frontend Scaffolding

Why this step: Match API contracts with shared TypeScript types early to reduce UI churn.

Frontend bootstrap

Optimized prompt (copy into Cursor)

- Scaffold Vite React-TS in /frontend with routing, layout shell, env-based API base URL, and typed API client.
- Create types mirroring backend schemas for Job and JobProblem.
- Keep styling simple and consistent — clean utility CSS; avoid generic purple AI themes.
- Wire a smoke API health indicator on the shell. Stop before full pages.

→ Pause: review output, approve, then continue to the next step.

Step 18: Build reusable Jobs Console UI primitives

Cursor topic: Component-Based Development

Why this step: Reusable primitives keep CRUD pages thin and consistent.

UI primitives

Optimized prompt (copy into Cursor)

- Create reusable components: PageHeader, StatusBadge, JobTypeBadge, DataTable scaffold, Modal, FormField, EmptyState, Toast.
- Map JobType labels to human text for the four pipelines (Epicor→GD WH, TXT→RPT, RPT→Fabric, Data Flow→Lake House).
- No page-level business logic inside primitives.
- Show a temporary /dev/ui usage route if helpful — remove if not wanted.

→ Pause: review output, approve, then continue to the next step.

Step 19: Build Dashboard (jobs overview)

Cursor topic: React Development — Dashboard

Why this step: Dashboard answers: what is running, failing, and which pipelines matter today.

Dashboard page

Optimized prompt (copy into Cursor)

- Build Dashboard with counts by status and by JobType, recent jobs, and unresolved problems count.
- Use real API data; loading/empty/error states required. No fake infinite widgets.
- One primary CTA: Create Job. Keep first viewport focused (not a dense ops wall).
- Confirm basic mobile usability.

→ Pause: review output, approve, then continue to the next step.

Step 20: Build Jobs List with search/filter/sort

Cursor topic: React Development — Jobs List

Why this step: List is the daily operator surface for the Organization Job Handler.

Jobs list

Optimized prompt (copy into Cursor)

- Build Jobs List page: columns name, type, source→target, status, updated_at, actions.
- Wire search, filter (type/status), sort, pagination to backend query params.
- Render the four example jobs clearly in type badges. Row click opens detail.
- Handle empty and error states. Do not implement edit form in this step.

→ Pause: review output, approve, then continue to the next step.

Step 21: Create Job form (Create)

Cursor topic: Forms & Validation

Why this step: Create form encodes domain constraints (JobType ↔ source/target) at the edge.

Create Job

Optimized prompt (copy into Cursor)

- Build Create Job form with client validation aligned to Pydantic rules.
- When JobType selected, auto-suggest/default source_system and target_system for the four pipelines.
- Submit via POST /jobs; show toast; navigate to detail/list on success; surface 422 field errors.
- No unrelated settings panels in this form.

→ Pause: review output, approve, then continue to the next step.

Step 22: Edit Job + Delete/Archive Job

Cursor topic: CRUD Operations — Update/Delete

Why this step: Complete the Jobs resource UX with safe destructive actions.

Edit & delete

Optimized prompt (copy into Cursor)

- Build Edit Job page/modal prefilled from GET /jobs/{id}; PUT/PATCH on save.
- Implement Delete or Archive with confirmation dialog; respect backend soft-delete policy from SRS.
- Disable illegal transitions if status rules exist. Refresh list/detail after mutations.
- Document UX rules only if non-obvious.

→ Pause: review output, approve, then continue to the next step.

Step 23: Job Problems UI (list, add, resolve)

Cursor topic: API Integration — nested Job Problems

Why this step: Operators need problem capture next to the job — this is a core FR, not a nice-to-have.

Problems UX

Optimized prompt (copy into Cursor)

- On Job Detail, show Problems table with severity, code, message, timestamps, resolve action.
- Add Problem form and Resolve flow calling backend endpoints.
- Filter unresolved by default. Show empty state "No open problems".
- Stop when CRUD for problems is usable end-to-end against local API.

→ Pause: review output, approve, then continue to the next step.

Step 24: Harden FE↔BE integration (context, errors, toasts)

Cursor topic: Context Management & API Consumption

Why this step: Centralize API client behavior so pages stay declarative.

Integration hardening

Optimized prompt (copy into Cursor)

- Centralize fetch/axios client: base URL, JSON headers, error normalization, optional auth header stub.
- Ensure consistent loading skeletons and toast notifications across Jobs and Problems.
- Verify CORS and env configuration for local dev. Fix any contract mismatches found.
- Produce a short integration checklist for QA.

→ Pause: review output, approve, then continue to the next step.

PHASE 5 — Pipeline Job Types & Status / Orchestration UX

Step 25: Model the four pipeline jobs as domain templates

Cursor topic: Feature Enhancement — Domain Job Templates

Why this step: The Excel ticket app ignored domain pipelines; here they are first-class.

Pipeline templates

Optimized prompt (copy into Cursor)

- Implement Job templates/presets for: (1) Epicor → GD Warehouse Sync, (2) TXT → RPT tables, (3) RPT → Fabric tables (Data Flow), (4) Data Flow → Lake House.
- Each template sets `job_type`, `source_system`, `target_system`, and a config JSON schema stub (paths, table names, watermark — metadata only for MVP).
- Add a "Create from template" UX entry point. Do not build real Epicor/Fabric connectors yet unless approved.
- Document what is simulated vs integrated.

→ Pause: review output, approve, then continue to the next step.

Step 26: Job run status timeline (MVP simulation)

Cursor topic: Feature Enhancement — Run status

Why this step: A minimal run history makes the console feel operational without overbuilding an orchestrator.

Run status MVP

Optimized prompt (copy into Cursor)

- If approved in SRS: add `job_runs` table/API or a status transition endpoint (queued→running→succeeded/failed).
- UI: show latest run status and a simple timeline on Job Detail.
- On failure, allow creating a linked `JobProblem` from the failed run message.
- Keep orchestration simulated locally (button "Simulate run") unless real workers exist. Ask before adding Celery/RQ.

→ Pause: review output, approve, then continue to the next step.

PHASE 6 — Quality: Review, Refactor, Tests, Debugging

Step 27: Full-codebase review focused on architecture

Cursor topic: AI Code Review

Why this step: Reviews catch coupling, missing indexes, and insecure defaults before tests paper over them.

Architecture review

Optimized prompt (copy into Cursor)

- Review the codebase against SRS and rules: layering, naming, dead code, secret leaks, N+1, missing validation.
- Produce findings as Severity Critical/Major/Minor with file references and recommended fixes.
- Do not mass-refactor yet — wait for prioritized fix list approval.

→ Pause: review output, approve, then continue to the next step.

Step 28: Targeted refactor of duplicates and boundaries

Cursor topic: Inline Edit & Refactoring

Why this step: PE-controlled refactors are safer than "clean everything".

Refactor pass

Optimized prompt (copy into Cursor)

- Refactor only approved items: extract shared API helpers, unify status badge mapping, remove duplicated validation.
- Prefer Inline Edit for single-file fixes; Agent for multi-file moves with a file list constraint.
- Keep behavior identical — no feature creep. Summarize before/after.

→ Pause: review output, approve, then continue to the next step.

Step 29: Naming, typing, and micro-optimizations

Cursor topic: Inline Editing / Clarity

Why this step: Clear names and types reduce future Agent hallucination.

Clarity pass

Optimized prompt (copy into Cursor)

- Rename vague symbols (data, temp, handleClick) to domain terms (job, jobProblem, pipelineType).
- Strengthen TypeScript types and Pydantic models where any remnants remain.
- Avoid premature memoization; follow project React guidance. Report diffs only.

→ Pause: review output, approve, then continue to the next step.

Step 30: Performance and query pass

Cursor topic: Performance Optimization

Why this step: Job lists and problem boards grow — design for pagination and indexes early.

Perf pass

Optimized prompt (copy into Cursor)

- Identify slow list endpoints and recommend query/index changes with EXPLAIN intuition.
- Ensure frontend lists do not over-fetch; confirm pagination works.
- Propose top 3 improvements only; implement after approval.

→ Pause: review output, approve, then continue to the next step.

Step 31: Generate backend unit & API tests

Cursor topic: Test Case Generation / Backend Testing

Why this step: API tests lock CRUD contracts for Jobs and Problems.

Backend tests

Optimized prompt (copy into Cursor)

- Add pytest API tests for Jobs and Job Problems CRUD, validation errors, and filters.
- Use a test database strategy (prefer Postgres test container if already in compose).
- Keep tests deterministic and fast. Show how to run them.

→ Pause: review output, approve, then continue to the next step.

Step 32: Generate frontend component/page tests

Cursor topic: Frontend Testing

Why this step: Critical forms and list filters deserve regression safety.

Frontend tests

Optimized prompt (copy into Cursor)

- Add Vitest/RTL tests for StatusBadge/JobTypeBadge, Create Job validation, and Jobs list filter behavior (mocked API).
- Do not chase 100% coverage — prioritize critical paths. Show how to run tests.

→ Pause: review output, approve, then continue to the next step.

Step 33: Deliberate bug-fix drill with Cursor

Cursor topic: AI-Assisted Debugging

Why this step: Practice hypothesis-driven debugging before production incidents.

Debug drill

Optimized prompt (copy into Cursor)

- Introduce a controlled bug (e.g., wrong filter query param) OR debug a real failing test.
- Use logs + failing test to locate root cause; fix with minimal diff; explain root cause in 5 lines.
- No speculative rewrites.

→ Pause: review output, approve, then continue to the next step.

Step 34: Logging, structured errors, and observability basics

Cursor topic: Error Handling & Logging

Why this step: Operational jobs need visible failures.

Observability basics

Optimized prompt (copy into Cursor)

- Ensure structured logging around job create/update and problem resolve.
- Map unexpected exceptions to safe 500 responses without leaking internals.
- Add correlation request id middleware if missing. Document log locations.

→ Pause: review output, approve, then continue to the next step.

PHASE 7 — Docs, Git Workflow, Docker, Deploy, CI/CD, Roadmap

Step 35: Generate OpenAPI-driven API documentation notes

Cursor topic: Documentation Generation

Why this step: FastAPI already exposes OpenAPI — capture human usage for operators/devs.

API docs

Optimized prompt (copy into Cursor)

- Ensure routes have summaries/tags; export notes in docs/api.md with auth/env prerequisites.
- Include examples for the four template jobs. Link to /docs Swagger UI.

→ Pause: review output, approve, then continue to the next step.

Step 36: Generate README for the monorepo

Cursor topic: Project Documentation

Why this step: README is the onboarding prompt for humans and future Agents.

README

Optimized prompt (copy into Cursor)

- Write root README: overview, architecture diagram, quickstart (Postgres, backend, frontend), env vars, scripts, test commands.
- Include domain glossary for JobType pipelines. Keep accurate — no aspirational features unmarked.

→ Pause: review output, approve, then continue to the next step.

Step 37: Git commit hygiene with Agent assistance

Cursor topic: Git Assistance

Why this step: Reviewable commits beat giant Agent dumps.

Commits

Optimized prompt (copy into Cursor)

- Group changes into logical commits (schema, backend jobs API, problems API, frontend list, etc.).
- Propose conventional commit messages. Wait for explicit approval before committing.
- Never commit .env or secrets.

→ Pause: review output, approve, then continue to the next step.

Step 38: Pull request description template

Cursor topic: Pull Request Support

Why this step: PR text should map to SRS stories and test plan.

PR body

Optimized prompt (copy into Cursor)

- Draft PR title/body with Summary, Test plan checklist (Jobs CRUD, Problems, templates, filters).
- List risks and screenshots placeholders. Do not push unless asked.

→ Pause: review output, approve, then continue to the next step.

Step 39: Pre-commit review of git diff

Cursor topic: Code Review with Git

Why this step: Last human+AI gate before sharing.

Diff review

Optimized prompt (copy into Cursor)

- Review git status/diff for secrets, debug junk, unfinished TODOs, and SRS gaps.
- Produce a go/no-go checklist. Fix only critical issues if approved.

→ Pause: review output, approve, then continue to the next step.

Step 40: Dockerize app + Postgres

Cursor topic: Containerization

Why this step: Compose makes the stack reproducible for the team.

Docker

Optimized prompt (copy into Cursor)

- Create Dockerfile for backend and frontend and docker-compose with PostgreSQL, API, web.
- Mount code for local iteration if useful. Document ports. No privileged containers.
- Do not push images. Verify compose config with a dry explanation of boot order.

→ Pause: review output, approve, then continue to the next step.

Step 41: Environment configuration and secrets policy

Cursor topic: Environment Management

Why this step: Separate config from code; keep examples committed.

Env policy

Optimized prompt (copy into Cursor)

- Finalize .env.example for backend/frontend; document each variable.
- Confirm secrets are gitignored. Warn about committing connection strings.

→ Pause: review output, approve, then continue to the next step.

Step 42: Deployment sketch for target environment

Cursor topic: Deployment

Why this step: Deployment stays a sketch until hosting choice is fixed.

Deploy notes

Optimized prompt (copy into Cursor)

- Given assumptions (e.g., single VM or cloud PaaS), outline deploy steps for API, web, and Postgres migrations.
- Include rollback notes for Alembic. Ask which host to target before writing vendor-specific IaC.

→ Pause: review output, approve, then continue to the next step.

Step 43: CI/CD workflow (GitHub Actions)

Cursor topic: DevOps Automation

Why this step: CI protects main from broken CRUD contracts.

CI pipeline

Optimized prompt (copy into Cursor)

- Create GitHub Actions workflow: lint/test backend, lint/test frontend, optional compose smoke.
- Keep it minimal and green locally-equivalent. No deploy-on-push unless approved.

→ Pause: review output, approve, then continue to the next step.

Step 44: Final end-to-end review

Cursor topic: End-to-End Code Review

Why this step: Close the loop against original FRs and pipeline examples.

E2E review

Optimized prompt (copy into Cursor)

- Trace each MVP FR to code + test. Verify four job templates exist and Problems CRUD works.
- List residual debt. Severity-ranked. No new features in this step.

→ Pause: review output, approve, then continue to the next step.

Step 45: Future enhancements roadmap

Cursor topic: Maintenance & Roadmap

Why this step: Capture real connectors and scheduling as post-MVP deliberately.

Roadmap

Optimized prompt (copy into Cursor)

- Propose phased roadmap: real Epicor connector, Fabric ingestion, Lake House loaders, scheduler/workers, RBAC, audit log, alerting.
- For each item: value, complexity, dependency. Save docs/roadmap.md.

→ Pause: review output, approve, then continue to the next step.

Approval gate

This PDF is the Cursor Steps plan only. Do not start generating application code until the human approves Phase order and step scope. Next action after approval: write/execute Step prompts in order, one approval gate at a time.

Job Handler · Senior Prompt Engineer Cursor Playbook · React + FastAPI + PostgreSQL